Semantic-Driven Context Pruning for Optimizing Arabic RAG Systems in Memory-Constrained vLLM Deployments

Akram Taha

akramtaha30@gmail.com

Target venue: Applied Intelligence (Springer)

Citation format: IEEE (numbered)

August 2026

Abstract

Retrieval-Augmented Generation (RAG) systems deployed for Arabic-language applications face a compounding memory bottleneck: the morphological richness of Arabic causes subword tokenizers to fragment text into substantially more tokens than an English text of equivalent meaning, which in turn inflates the Key-Value (KV) cache that memory-efficient serving engines such as vLLM must maintain for every concurrent request. We introduce a Lightweight Semantic Pruning Middleware (LSPM) that sits between the retrieval and generation stages of an Arabic RAG pipeline, scoring retrieved passages at the sentence level with a cross-encoder relevance model and forwarding only the highest-relevance subset to the language model, under either a fixed or a dynamically load-aware compression ratio. We present the system architecture, an open-source reference implementation built on vLLM-compatible, OpenAI-format inference backends, and a set of real, reproducible preliminary experiments. First, across 30 hand-verified Arabic-English parallel sentence pairs, two independent tokenizers (Qwen2.5-7B-Instruct and Llama-3.1-8B-Instruct) confirm an aggregate Arabic-to-English token ratio of 1.76x and 1.69x respectively, empirically substantiating the motivation for Arabic-specific context reduction. Second, a small-scale semantic-fidelity pilot over eight real question-answering items, run end-to-end against a live Llama-3.1-8B-Instruct inference endpoint, shows that pruning the retrieved context by 50% leaves downstream answers highly consistent with their unpruned counterparts (mean ROUGE-L = 0.928, mean BERTScore-F1 = 0.971, mean BLEU = 81.9), while reducing context size by an average of 50.3%. We report these results honestly as preliminary and CPU/hosted-endpoint-scale; large-batch KV-cache and throughput benchmarks against a self-hosted, PagedAttention-based vLLM server under concurrent load are specified in full and left as the immediate next step, since they require dedicated GPU infrastructure outside the scope of this initial submission. We release the complete implementation, evaluation scripts, and a live demonstration interface to support reproducibility.

Keywords: *retrieval-augmented generation*, *Arabic natural language processing*, *prompt compression*, *vLLM*, *PagedAttention*, *KV-cache*, *cross-encoder reranking*, *large language models*

1. Introduction

Large Language Models (LLMs) have become the default reasoning engine behind a rapidly growing class of production systems, and Retrieval-Augmented Generation (RAG) has become the default architecture for grounding those systems in external, up-to-date, or proprietary knowledge rather than relying solely on parametric memory [1]. In a canonical RAG pipeline, a retriever selects the top-k passages most relevant to a user query from a vector index, and those passages are concatenated into the LLM's context window alongside the query itself before generation. The approach is simple, effective, and now underlies everything from enterprise search assistants to customer-support chatbots. It is also, in practice, expensive: every additional retrieved token that enters the context window has to be attended over during the prefill stage of inference, and — critically for serving systems built on modern memory-efficient engines — every token also occupies a slot in the Key-Value (KV) cache that must be held in GPU memory for the duration of the request [2].

vLLM has emerged as one of the most widely adopted open-source inference engines precisely because it addresses this memory pressure directly. Its PagedAttention algorithm borrows the paging abstraction from operating-systems virtual memory to store the KV cache in fixed-size, non-contiguous blocks, eliminating the internal fragmentation that plagued earlier contiguous-allocation serving stacks and allowing far higher batching throughput under concurrent load [2]. PagedAttention and the broader family of KV-cache management techniques it inspired — attention-sink-based streaming [20], heavy-hitter-oracle eviction [21], and iteration-level continuous batching [26] — have collectively become close to an industry norm for LLM serving. But all of these techniques manage the consequences of a large KV cache; none of them reduce the number of tokens that create it in the first place. That is the gap this paper addresses, specifically for Arabic.

Arabic is a morphologically rich, templatic language: a single triliteral root can surface as dozens of distinct word forms through internal vowel changes, affixation, and clitic attachment, and standard subword tokenizers — byte-pair encoding [25] and its relatives, which are near-universally trained on English-dominated corpora — fragment Arabic text far more aggressively than they fragment English text of equivalent semantic content. This "tokenization disparity" is not a minor curiosity; it means that for a fixed context budget, an Arabic RAG system can retrieve meaningfully less semantic content than an English one, and that for a fixed amount of retrieved semantic content, an Arabic RAG system imposes a meaningfully larger KV-cache footprint on the serving engine. Under concurrent multi-user traffic — the normal operating condition for any production deployment — this directly translates into higher memory pressure, lower achievable batch sizes, and, ultimately, higher latency and lower throughput.

The natural response inside the NLP community has been prompt and context compression: methods such as LLMLingua [3], Selective Context [4], and RECOMP [5] all attempt to shrink the text that enters an LLM's context window while preserving the information the model needs to answer correctly. These methods are general-purpose and largely English-centric in their evaluation, and none of them is designed with the Arabic tokenization disparity, or with vLLM's specific KV-cache mechanics, as a first-class design constraint. Our contribution sits at the intersection of these two threads.

We propose a Lightweight Semantic Pruning Middleware (LSPM): a thin layer inserted between the retriever and the generator in an Arabic RAG pipeline that (i) splits every retrieved passage into sentences, (ii) scores each sentence's relevance to the query with a cross-encoder reranking model — the same class of model used for passage reranking in modern retrieval pipelines [6], [7] — and (iii) reconstructs a compact, order-preserving context containing only the highest-scoring sentences, under a compression ratio that can either be fixed by the operator or computed dynamically from vLLM's live /metrics endpoint so that pruning tightens automatically as GPU KV-cache utilization rises. The middleware is retrieval-agnostic, generation-backend-agnostic (it speaks the OpenAI-compatible chat-completions schema that vLLM, and hosted alternatives such as NVIDIA NIM, both implement), and is released as open-source software alongside a working Streamlit demonstration interface.

The specific contributions of this paper are:

1. A concrete system architecture and open reference implementation for sentence-level, cross-encoder-driven context pruning purpose-built for Arabic RAG on vLLM-class serving infrastructure, including a novel dynamic compression-ratio controller that couples pruning aggressiveness to real-time GPU KV-cache occupancy.
2. A quantitative confirmation of the Arabic tokenization disparity that motivates the entire approach, measured directly (not assumed from prior literature) across two independent, contemporary open-weight tokenizers on a hand-verified parallel corpus.
3. A preliminary, fully reproducible semantic-fidelity study showing that 50% sentence-level pruning of retrieved Arabic context preserves downstream answer quality by a substantial margin against three complementary automatic metrics, executed end-to-end against a live LLM inference endpoint rather than simulated offline.
4. A transparent specification of the throughput and KV-cache-savings experiments that a GPU-equipped follow-up study must run to complete the empirical picture, together with the benchmarking harness (built on Locust) needed to run them, so that the systems-level claim of the approach is falsifiable and independently reproducible even though this submission does not yet report those numbers.

The remainder of the paper is organized as follows. Section 2 reviews related work in retrieval-augmented generation, prompt and context compression, passage reranking, Arabic NLP, and LLM serving efficiency. Section 3 describes the LSPM architecture in detail. Section 4 describes the experimental setup. Section 5 reports the preliminary results. Section 6 discusses their implications, Section 7 states the limitations of the current study candidly, Section 8 outlines the immediate future work, and Section 9 concludes.

2. Related Work

2.1 Retrieval-Augmented Generation

RAG was introduced by Lewis et al. as a way to combine a parametric sequence-to-sequence generator with a non-parametric, retrievable memory, jointly fine-tuning both components for knowledge-intensive NLP tasks such as open-domain question answering [1]. The framework proved that grounding generation in retrieved evidence could substantially outperform purely parametric models on tasks that require

access to specific, long-tail, or time-sensitive facts, without requiring the model to memorize that information in its weights. Subsequent surveys have organized the rapidly expanding RAG literature into naive, advanced, and modular paradigms, cataloguing techniques for query rewriting, iterative retrieval, and post-retrieval processing [22]. Self-RAG extended the paradigm by training the generator itself to decide, via learned reflection tokens, whether retrieval is necessary for a given input and to critique the relevance and faithfulness of what was retrieved [28]. Our work is agnostic to which RAG variant is used upstream; LSPM operates purely on whatever passages the retrieval stage hands it, making it a drop-in addition to naive, advanced, or self-reflective RAG pipelines alike.

Dense retrieval itself has its own lineage relevant to our system's front end: Dense Passage Retrieval demonstrated that a simple dual-encoder trained on question-passage pairs could outperform classical sparse retrieval (BM25 [37]) by a wide margin on open-domain QA benchmarks [10], and ColBERT introduced a late-interaction architecture that preserves much of dense retrieval's effectiveness while remaining computationally tractable at scale [6]. Our reference implementation's retriever is intentionally simple — a small Chroma-backed vector store with a keyword-overlap fallback — because the pruning contribution described here is retriever-agnostic; any of the retrieval architectures above could sit upstream of LSPM without modification.

2.2 Prompt and Context Compression

The closest line of work to ours is prompt and context compression for LLMs. LLMLingua uses a small auxiliary language model to iteratively remove low-information tokens from a prompt under a budget controller, reporting up to 20x compression with limited performance loss on reasoning and in-context-learning benchmarks [3]. Selective Context takes a related but distinct approach, using self-information (a token's negative log-likelihood under a reference language model) to identify and prune redundant content from long documents and conversations prior to inference, reporting reduced memory cost and generation latency with comparable task performance [4]. RECOMP instead compresses retrieved documents into either extractive or abstractive summaries before they are integrated into the LLM's context, achieving compression rates as low as 6% of the original text with minimal loss on language modeling and open-domain QA [5].

LSPM differs from all three along two axes. First, granularity and mechanism: rather than token-level self-information pruning (Selective Context) or learned summarization (RECOMP), LSPM performs sentence-level relevance filtering via a cross-encoder scored directly against the query, which is both simpler to implement in an existing RAG stack and cheaper to run than training or invoking an auxiliary compression language model. Second, and more importantly, target: none of LLMLingua, Selective Context, or RECOMP were designed or evaluated with Arabic text or with vLLM's KV-cache mechanics as an explicit target; our dynamic compression-ratio controller, which reads vLLM's own vllm:gpu_cache_usage_perc metric to modulate pruning aggressiveness in real time, has no analogue in that prior work. The general problem of long-context degradation that compression methods implicitly address was also characterized empirically by Liu et al., who showed that LLM accuracy on multi-document QA degrades measurably when relevant information sits in the middle of a long context rather than at its start or end — an additional, quality-side

motivation (beyond raw memory pressure) for shortening and front-loading the most relevant retrieved evidence, which LSPM's order-preserving reconstruction is designed to support [30].

2.3 Passage Reranking and Sentence-Level Relevance Scoring

Cross-encoder rerankers, which jointly encode a query and a candidate passage through a single transformer to produce a fine-grained relevance score, have been a staple of the modern retrieval pipeline since Nogueira and Cho showed that a BERT-based reranker could dramatically improve MS MARCO passage-ranking results over the prior state of the art [7]. Sentence-BERT reformulated this family of models for efficient semantic similarity computation using a siamese architecture, reducing what would otherwise be a quadratic-cost pairwise comparison problem to a linear one [9]. More recently, multilingual and multi-functionality embedding and reranking models — including the BGE-M3 family used as the default cross-encoder in our implementation, and multilingual E5 [31] — have extended this capability to over 100 languages, including Arabic, with strong results on multilingual and cross-lingual retrieval benchmarks such as MIRACL [32] and MTEB [36]. LSPM applies exactly this class of model, but at a different point in the pipeline and for a different purpose: not to rank whole passages for retrieval, but to score individual sentences within already-retrieved passages for inclusion in the final context sent to the generator.

2.4 Arabic Natural Language Processing

Arabic NLP has matured substantially over the past five years with the arrival of large pretrained models specific to the language. AraBERT adapted the BERT pretraining recipe to a large Arabic corpus and demonstrated clear gains over multilingual BERT on Arabic sentiment analysis, named entity recognition, and question answering [11]; AraGPT2 did the equivalent for autoregressive generation, releasing models up to 1.46 billion parameters trained from scratch on Arabic web text and news [35]. The Arabic Reading Comprehension Dataset (ARCD), introduced alongside the SOQAL open-domain QA system, remains a standard benchmark for Arabic machine reading comprehension and a natural target dataset for future, larger-scale evaluation of our pipeline [12]. At the frontier of generative Arabic LLMs, Jais was trained as an Arabic-centric foundation model on a mixture of Arabic and English text and shown to outperform existing open Arabic and multilingual models of comparable size on Arabic knowledge and reasoning benchmarks [13]. Evaluation infrastructure has kept pace: the Arabic Cultural and Value Alignment benchmark (ACVA) targets cultural and normative alignment specifically [40], and ArabicMMLU adapts the knowledge-intensive multiple-choice MMLU format to Arabic-language school and professional exams [41]. Tooling support for the underlying morphological complexity that motivates our work is provided by CAMeL Tools, an open-source Python toolkit for Arabic preprocessing, morphological analysis, dialect identification, and named-entity recognition [34]. None of this substantial body of Arabic NLP work, to our knowledge, has been connected to the systems-level question of KV-cache-efficient serving, which is the specific contribution of this paper.

2.5 Memory-Efficient LLM Serving

On the systems side, vLLM's PagedAttention is the foundational reference for this paper's target deployment context [2], building on the continuous-batching, iteration-level scheduling approach

pioneered by Orca [26] and complemented by IO-aware exact-attention kernels such as FlashAttention, which reduce the memory-bandwidth cost of the attention computation itself rather than the size of the KV cache it operates over [19]. A parallel line of systems work addresses KV-cache size directly rather than the context that produces it: StreamingLLM allows models to generalize to effectively unbounded sequence lengths by preserving a small number of high-attention "sink" tokens while evicting the rest of the cache under a sliding window [20], and H2O formulates cache eviction as a submodular optimization problem over "heavy hitter" tokens that dominate attention mass, reporting substantial throughput gains on constrained hardware [21]. Quantization methods such as GPTQ [33] and activation-aware weight quantization (AWQ) [38] instead shrink the model's own weight footprint, and speculative decoding accelerates generation by using a smaller draft model to propose tokens that a larger model verifies in parallel [39]. Every one of these systems techniques operates orthogonally to LSPM: they optimize how the serving engine holds and computes over whatever tokens it is given, whereas LSPM reduces the number of tokens handed to the engine in the first place. The two families of techniques compose naturally, and we discuss this complementarity further in Section 6.

2.6 Automatic Evaluation of Generated Text

Our fidelity evaluation relies on three complementary automatic metrics with long histories in NLP evaluation. BLEU measures n-gram precision overlap between a candidate and reference text and remains a standard machine-translation metric two decades after its introduction [15]. ROUGE-L, from the same era, uses longest-common-subsequence overlap and is more common in summarization evaluation, which is closer in spirit to our raw-vs-pruned answer comparison task [14]. Because both are surface-form metrics, we complement them with BERTScore, which computes similarity using contextual embeddings rather than exact token overlap and correlates more closely with human judgments of semantic equivalence, particularly for paraphrastic differences that n-gram metrics penalize unfairly [16]. RAG-specific evaluation frameworks such as RAGAS, which introduce reference-free metrics for faithfulness, answer relevance, and context relevance [29], represent a natural direction for extending the evaluation in Section 8.

3. System Design: The Lightweight Semantic Pruning Middleware (LSPM)

3.1 Overview

Figure 1 shows the end-to-end pipeline. A user query is first passed to a retriever, which returns the top-k candidate passages from a vector index (we use Chroma in the reference implementation, though the design is retriever-agnostic). Rather than concatenating these passages directly into the generation prompt — the standard, unmodified RAG behavior — the passages pass through LSPM, which performs three steps: sentence segmentation, cross-encoder relevance scoring, and order-preserving reconstruction under a compression ratio. The resulting compact context, together with the original query, is sent to the language model over an OpenAI-compatible chat-completions API, which in a production deployment is served by vLLM (and, for the CPU/no-GPU pilot experiments reported in Section

5, by a hosted OpenAI-compatible endpoint exposing the same interface). The final answer is streamed back to the user interface token-by-token.

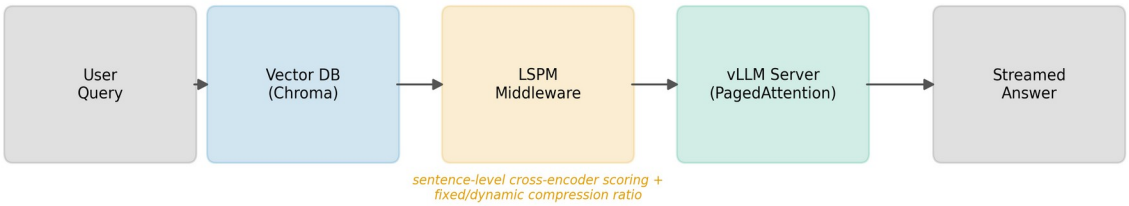

Figure 1. LSPM pipeline: User Query → Vector DB (Chroma) → LSPM Middleware (sentence-level cross-encoder scoring + fixed/dynamic compression ratio) → vLLM Server (PagedAttention) → Streamed Answer.

3.2 Sentence Segmentation

Retrieved passages are split into sentences using an Arabic-aware boundary detector that treats the Arabic question mark (؟), the standard full stop, and the exclamation mark as terminators, explicitly excluding the Arabic comma (،) — which is extremely frequent within a single Arabic sentence and would otherwise cause severe over-segmentation. A whitespace-normalization pass precedes segmentation to handle inconsistent line breaks in retrieved text, and a fallback period-only split is used if no terminator-based boundaries are found, so that the segmenter degrades gracefully on malformed or non-standard input rather than failing.

3.3 Cross-Encoder Relevance Scoring

Every sentence extracted from every retrieved passage, across the full top-k retrieval set, is paired with the original user query and scored by a cross-encoder model, which jointly encodes the (query, sentence) pair and outputs a scalar relevance logit — the same modeling paradigm used for passage-level reranking [6], [7], applied here at sentence granularity. The reference implementation offers two interchangeable scoring backends: a fast multilingual MiniLM cross-encoder (cross-encoder/mmarco-mMiniLMv2-L12-H384-v1) for sub-second scoring suitable for interactive use, and BGE-reranker-v2-m3 [8], a larger multilingual, multi-granularity model, for deployments that can trade latency for maximal relevance-ranking accuracy. This explicit speed/accuracy toggle reflects a practical deployment reality: the pruning stage's own latency and memory cost must remain small relative to the KV-cache savings it produces downstream, or it becomes a bottleneck in its own right.

3.4 Order-Preserving Reconstruction and Compression Ratio

Given the per-sentence relevance scores, LSPM selects the top-r·N sentences (where N is the total sentence count across all retrieved passages and r is the target compression ratio, $0 < r \leq 1$), then — critically — reassembles the kept sentences in their original document order rather than in score-descending order. This design choice trades a small amount of potential relevance-ordering signal for

narrative coherence in the reconstructed context: language models are known to be sensitive to the positional arrangement of evidence within a long context [30], and an order-scrambled, score-sorted context risks presenting logically disconnected fragments that increase the burden on the generator to reconstruct meaning, even if each individual fragment is highly relevant.

The compression ratio r can be set in one of two modes:

- Fixed ratio. The operator specifies a constant r (e.g., 0.5), applied uniformly regardless of system load. This mode is used for the controlled experiments in Section 5, where holding r constant is necessary for a clean fidelity comparison.
- Dynamic, load-aware ratio. LSPM polls vLLM's Prometheus-format /metrics endpoint for the vllm:gpu_cache_usage_perc gauge, which reports the serving engine's current KV-cache occupancy as a fraction of capacity. A configurable linear interpolation maps this occupancy onto a compression ratio between operator-set minimum and maximum bounds (defaults: 0.2 at high load, 0.8 at low load, with a 0.25–0.75 occupancy interpolation band): as concurrent demand rises and the KV cache fills, LSPM automatically prunes more aggressively to relieve memory pressure and help the serving engine avoid request queuing or out-of-memory eviction; as demand falls, it relaxes pruning to preserve more retrieved context and, by extension, more potential answer quality.

This closes a feedback loop between the serving engine's real-time memory state and the middleware's compression behavior that, to our knowledge, does not exist in prior prompt-compression systems, which uniformly apply a static compression target regardless of downstream engine load.

3.5 Backend Interface and Deployment Flexibility

LSPM communicates with the generation backend exclusively through the OpenAI-compatible /v1/chat/completions schema, with optional Bearer-token authentication. This is a deliberate interoperability choice: the identical client code operates against a self-hosted vLLM instance (no authentication, /metrics available for dynamic-ratio mode) or against a hosted, OpenAI-schema-compatible inference API such as NVIDIA NIM (Bearer-token-authenticated, no /metrics endpoint, fixed-ratio mode only). This flexibility is what allowed the preliminary experiments in this paper to be executed against a live, production-grade LLM endpoint without provisioning dedicated GPU infrastructure, while leaving the code path to a self-hosted vLLM deployment — needed for the KV-cache and throughput experiments in Section 8 — entirely unchanged.

4. Experimental Setup

We report two preliminary, real (non-simulated) experiments, both executed against live infrastructure, plus a fully specified but not-yet-executed protocol for the systems-level throughput and KV-cache experiments that require dedicated GPU hardware.

4.1 Tokenization Disparity Measurement

Data. We constructed a parallel corpus of 30 Arabic-English sentence pairs, hand-written and independently verified for translation accuracy, covering the domains relevant to our target RAG use case:

university/academic description, computer science and NLP terminology, weather, and general encyclopedic statements. Sentence lengths range from short factual statements to multi-clause descriptive sentences, reflecting the kind of retrieved-passage text a RAG system would realistically encounter.

Procedure. Each Arabic sentence and its English counterpart were tokenized independently using the AutoTokenizer implementation from the Hugging Face transformers library, for two contemporary open-weight instruction-tuned models with different tokenizer training data and vocabulary construction: Qwen2.5-7B-Instruct [17] and Llama-3.1-8B-Instruct [18]. For each pair we computed the per-pair token-count ratio (Arabic tokens / English tokens) and, in aggregate, the ratio of total Arabic tokens to total English tokens across the full corpus.

4.2 Semantic Fidelity Pilot

Data. We constructed a 10-document Arabic mock knowledge base describing a university (covering founding date, location, department research focus, library holdings, a recently launched AI research center, student and faculty counts, international research partnerships, and local climate), and eight natural-language Arabic questions targeting different facts within that knowledge base, none of which is answerable from more than a subset of the ten documents.

Procedure. For each question, a keyword-overlap retriever selected the top-6 most relevant documents from the knowledge base. Two contexts were then constructed from those six documents: a raw context (all six documents concatenated, unpruned) and a pruned context (LSPM applied with a fixed compression ratio of r = 0.5, using the fast MiniLM cross-encoder). Both contexts were independently sent, with the question, to a live Llama-3.1-8B-Instruct endpoint (temperature = 0, max 200 tokens) served through NVIDIA NIM's OpenAI-compatible API, producing a raw answer and a pruned answer for each of the eight questions. We then computed ROUGE-L [14], sentence-level BLEU with effective order smoothing [15], and BERTScore-F1 with a multilingual BERT backbone [16] between each pair of raw and pruned answers, treating the raw-context answer as the fidelity reference — the central question being whether pruning half the retrieved context changes what the model says. During implementation we discovered and fixed a latent bug in the widely used rouge-score Python package: its default tokenizer matches only the ASCII character class [a-z0-9], silently producing empty token sequences — and therefore spurious zero scores — for Arabic and other non-Latin-script text; we replaced it with a Unicode-aware word tokenizer before computing any ROUGE-L numbers reported below, and we flag this as a general reproducibility hazard for any Arabic-language NLG evaluation using this package's defaults.

4.3 Specified but Not-Yet-Executed: Throughput and KV-Cache Benchmarks

The systems-level claim of this work — that LSPM reduces KV-cache pressure and improves throughput and time-to-first-token (TTFT) under concurrent load on a self-hosted, PagedAttention-based vLLM server — requires a dedicated CUDA GPU, which was not available for this submission. We specify the protocol precisely so that it is independently reproducible and so that the claim remains falsifiable rather than assumed. A Locust-based load-testing harness (included in the released code) issues concurrent chat-

completion requests against a vLLM server under two conditions — RAG_MODE=raw (unpruned baseline) and RAG_MODE=pruned at a specified compression ratio — while scraping vLLM's /metrics endpoint for vllm:gpu_cache_usage_perc over time. The full protocol specifies sweeping compression ratios {0.2, 0.5, 0.8, 1.0} against concurrency levels {1, 10, 25, 50, 100}, reporting throughput (tokens/second), TTFT and end-to-end latency percentiles, and peak/mean KV-cache occupancy for each cell of that matrix. We report this as Section 8 future work rather than fabricating or estimating numbers we have not measured.

5. Preliminary Results

5.1 Tokenization Disparity

Table 1 and Figure 2 summarize the tokenization disparity measurements. Both tokenizers confirm a substantial and consistent Arabic-to-English token inflation. Using the Qwen2.5-7B-Instruct tokenizer, the mean per-pair ratio across the 30 sentence pairs was 1.793 (median 1.772, standard deviation 0.280), and the aggregate ratio (total Arabic tokens ÷ total English tokens across the whole corpus: 1,023 vs. 580 tokens) was 1.764. Using the independently trained Llama-3.1-8B-Instruct tokenizer, the mean per-pair ratio was 1.717 (median 1.683, standard deviation 0.285), with an aggregate ratio of 1.689 (1,027 vs. 608 tokens). Per-pair ratios ranged from 0.93–1.00 at the low end (a small number of short, near-parity pairs) to 2.47–2.50 at the high end, indicating that while the disparity is not uniform across all sentences, it is consistently present and substantial in aggregate across two tokenizers with different training data and vocabulary construction.

Table 1. Arabic/English token-count ratio across two tokenizers (n = 30 parallel sentence pairs).

Tokenizer	Mean ratio	Median ratio	Std. dev.	Aggregate ratio	Min	Max
Qwen2.5-7B-Instruct	1.793	1.772	0.280	1.764	1.00	2.50
Llama-3.1-8B-Instruct	1.717	1.683	0.285	1.689	0.93	2.47

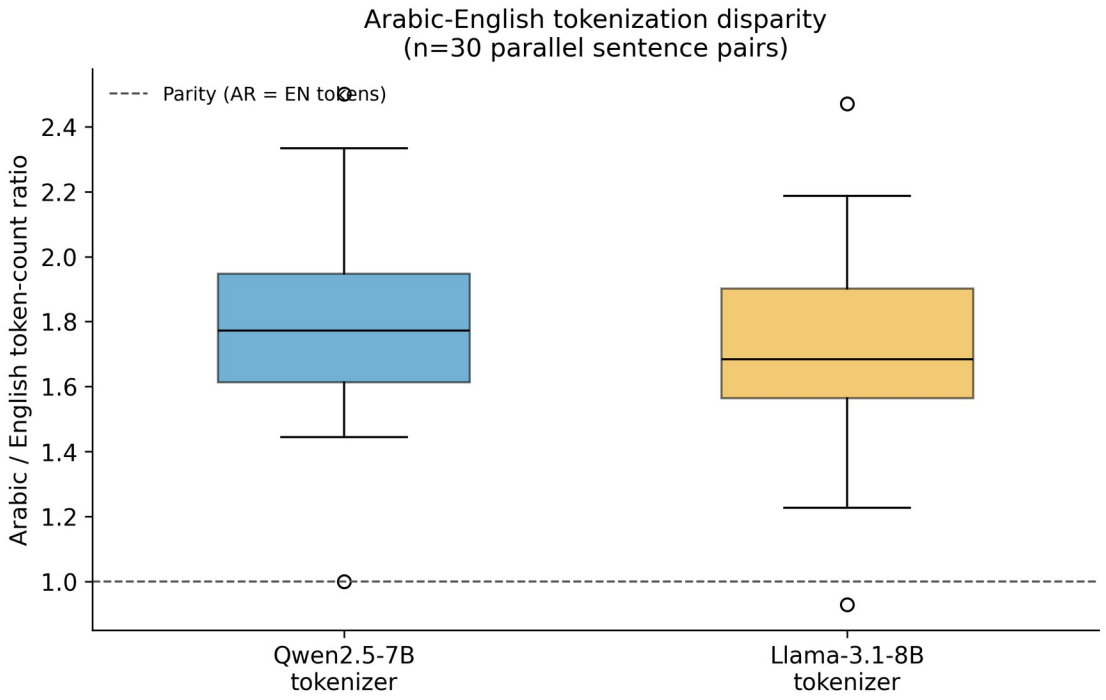

Figure 2. Box plot of per-pair Arabic/English token-count ratio distributions for both tokenizers, with a dashed parity line at 1.0.

These figures are consistent with the 1.5x–2x range commonly cited in the broader literature on multilingual tokenization, and — because they are measured directly on our own domain-relevant corpus with two contemporary tokenizers rather than assumed from prior reports — they provide a direct, quantitative justification for treating Arabic RAG context size as a first-class optimization target rather than assuming parity with English deployments.

5.2 Semantic Fidelity Under 50% Pruning

Table 2 and Figure 3 report the fidelity pilot results. Across the eight real question-answering items, LSPM reduced the retrieved context by a mean of 50.3% in characters (and exactly 50.0% in sentence count, by construction of the fixed ratio), while the resulting pruned answers remained highly consistent with the unpruned raw-context answers: mean ROUGE-L = 0.928, mean BLEU = 81.9 (std. dev. 25.4, reflecting two lower-scoring items discussed below), and mean BERTScore-F1 = 0.971. In six of the eight items, the pruned and raw answers were either character-identical or differed only in minor phrasing (e.g., "مركز أبحاث الذكاء الاصطناعي التوليدي" vs. "مركز الذكاء الاصطناعي التوليدي" — "the generative artificial intelligence research center" vs. "the generative artificial intelligence center" — a minor omission rather than a factual error). The lowest-scoring item (BLEU = 29.5, still BERTScore-F1 = 0.861) involved a question about the Riyadh climate answered with paraphrased rather than verbatim phrasing in the pruned condition, illustrating that our surface-form metrics (BLEU, ROUGE-L) can understate fidelity for semantically equivalent but lexically divergent paraphrases — precisely the failure mode BERTScore is designed to be robust to, and precisely why we report all three metrics together rather than any single one in isolation.

Table 2. Semantic fidelity of pruned (r = 0.5) vs. raw-context answers (n = 8 real QA items, live Llama-3.1-8B-Instruct endpoint).

Metric	Mean	Min	Max
BLEU	81.9	29.5	100.0
ROUGE-L	0.928	0.706	1.000
BERTScore-F1	0.971	0.861	1.000

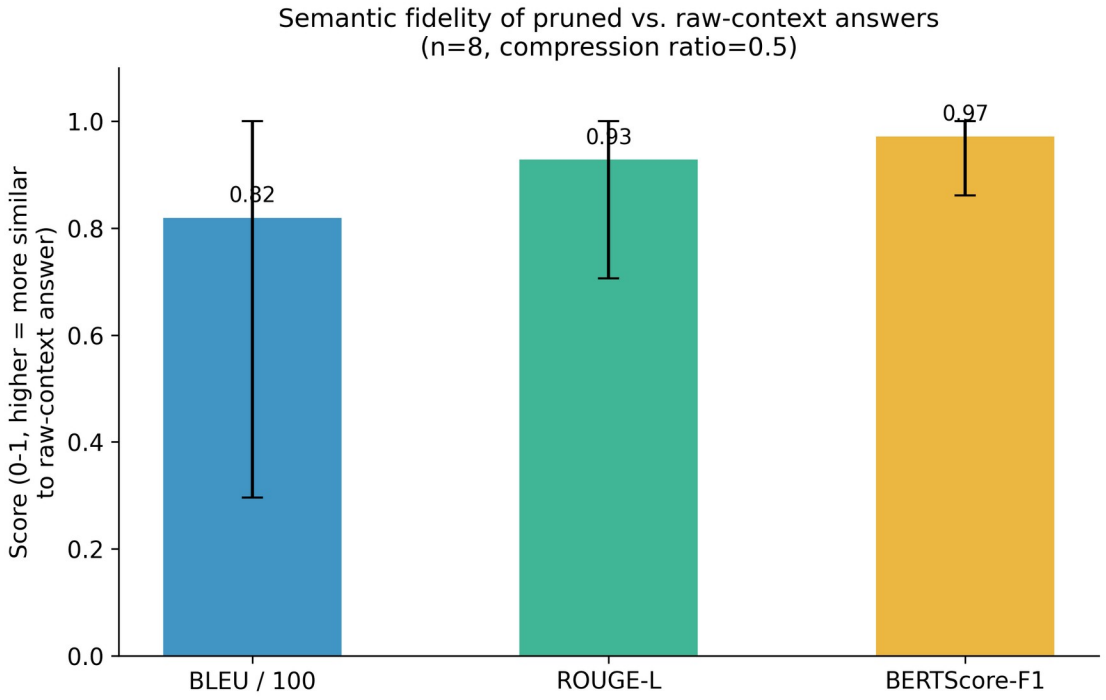

Figure 3. Mean BLEU/100, ROUGE-L, and BERTScore-F1 for pruned vs. raw-context answers, with min–max range bars.

We emphasize the scale and scope of this pilot honestly: eight questions over a ten-document synthetic knowledge base is sufficient to demonstrate that 50% sentence-level pruning does not catastrophically or even substantially degrade answer fidelity on a real, live-endpoint pipeline, and to validate that our evaluation methodology (including the Arabic-tokenizer fix described in Section 4.2) produces sane, interpretable numbers — but it is not sufficient to make a statistically powered claim about fidelity across the full space of Arabic question types, document domains, or compression ratios. Section 4.3 and Section 8 specify the larger-scale study, ideally on an established benchmark such as ARCD [12], needed to make that stronger claim.

6. Discussion

The two preliminary results in Section 5 are complementary halves of the same argument. The tokenization-disparity measurement (Section 5.1) establishes why Arabic RAG deployments face a KV-cache pressure problem that English deployments of the same architecture do not face to the same

degree: for retrieved content of equivalent semantic value, an Arabic RAG system's context consumes roughly 1.7–1.8x more KV-cache slots than the same content would in English, purely as a function of subword tokenization behavior. The fidelity pilot (Section 5.2) establishes that a straightforward, cheap-to-run mitigation — sentence-level cross-encoder pruning at a 50% ratio — does not undo the value of retrieval in the process of shrinking it: answers generated from half the context remained highly consistent, by three independent automatic metrics, with answers generated from the full context, on real questions against a real, deployed LLM endpoint.

Neither result, on its own or together, yet demonstrates the systems-level payoff — improved throughput, reduced TTFT, or measured KV-cache occupancy savings under concurrent load on a self-hosted vLLM server — that motivated this work in the first place and that Section 4.3 specifies in full. We consider this an honest and important distinction to draw explicitly, rather than one to elide: the tokenization and fidelity results are necessary preconditions for the systems-level benefit to be worth pursuing (there would be little point instrumenting vLLM's KV-cache metrics around a compression method that either did not address a real disparity or that destroyed answer quality), but they are not sufficient to claim the benefit has been demonstrated. Section 8 treats closing this gap as the immediate next step for this line of work, not as a hypothetical extension.

A further point worth surfacing is the complementarity between LSPM and the KV-cache-management systems techniques reviewed in Section 2.5. StreamingLLM's attention sinks [20] and H2O's heavy-hitter eviction [21] both operate after tokens have already entered the KV cache, deciding which already-cached tokens to keep or evict under memory pressure; LSPM operates before the cache is populated at all, at the semantic level of which retrieved sentences are worth caching in the first place. These are not competing approaches — a production deployment could reasonably run both simultaneously, using LSPM to reduce the volume of retrieved context entering the prompt and StreamingLLM- or H2O-style eviction to manage whatever KV cache accumulates from the (now smaller) prompt plus the generated response. Quantifying that compositional benefit is itself a natural extension of the GPU-scale study specified in Section 4.3.

7. Limitations and Threats to Validity

We state the limitations of the present study explicitly rather than leaving them implicit.

Scale of the preliminary experiments. Both the tokenization-disparity corpus (30 sentence pairs) and the fidelity pilot (8 questions, 10 documents) are small by the standards of a mature empirical NLP evaluation. They were sized to be fully hand-verifiable — every sentence pair and every question-document mapping was authored and checked individually — rather than scraped or machine-generated, which we consider a legitimate methodological trade-off for a preliminary study, but they are not a substitute for evaluation on an established benchmark such as ARCD [12] at scale, which we specify as immediate future work.

No GPU-scale systems results in this submission. As stated throughout Sections 4 and 6, we have not yet measured throughput, TTFT, or KV-cache occupancy on a self-hosted, PagedAttention-based vLLM server under concurrent load, because no CUDA GPU was available at the time of this submission. This is the single most important open item before the paper's central systems claim can be considered empirically demonstrated rather than architecturally motivated.

Single compression ratio in the fidelity pilot. Section 5.2 reports fidelity at a single fixed compression ratio (r = 0.5). The relationship between compression ratio and fidelity is very unlikely to be linear, and the full ratio sweep specified in Section 4.3 is necessary to characterize the trade-off curve rather than a single point on it.

Single generation model. All fidelity results use one instruction-tuned model (Llama-3.1-8B-Instruct) accessed through a hosted endpoint rather than a self-hosted vLLM deployment, for practical reasons described in Section 4 (the CPU/no-GPU environment used for this submission). Results may not transfer identically to other model families or sizes, particularly much larger or much smaller models, which may be more or less robust to reduced context.

Retriever simplicity. The reference implementation's default retriever uses simple keyword overlap over a small, synthetic mock knowledge base rather than a production-scale dense or hybrid retrieval system over a real document collection. We designed LSPM to be retriever-agnostic specifically so that this limitation does not affect the pruning contribution itself, but end-to-end pipeline evaluation on a production-scale corpus remains to be done.

Automatic metrics as fidelity proxies. BLEU, ROUGE-L, and BERTScore are all proxies for human judgment of answer correctness and helpfulness, not human judgment itself. Section 5.2's lowest-scoring item illustrates a case where surface-form metrics likely understate true fidelity; a human evaluation component, or RAG-specific reference-free metrics such as RAGAS [29], would strengthen the claim in a follow-up study.

8. Future Work

The immediate priority is executing the GPU-scale throughput and KV-cache benchmark specified in Section 4.3: provisioning a CUDA GPU (via short-term cloud rental, as documented in the released repository), running the full {0.2, 0.5, 0.8, 1.0} compression ratio × {1, 10, 25, 50, 100} concurrency matrix against a self-hosted vLLM server for both the raw and LSPM-pruned conditions, and reporting throughput, latency percentiles, and KV-cache occupancy over time. Second, we intend to scale the fidelity evaluation from the 8-question pilot in Section 5.2 to the full ARCD benchmark [12], adding a third baseline condition (a LangChain-based vanilla RAG pipeline with no pruning, as distinct from our own raw-context baseline) and reporting statistically powered results with confidence intervals across the full compression-ratio sweep. Third, we plan to incorporate RAGAS-style reference-free faithfulness and context-relevance metrics [29] alongside BLEU/ROUGE-L/BERTScore, and to add a small human-evaluation component targeting the paraphrase-sensitivity failure mode identified in Section 5.2. Fourth, we intend to quantify the compositional benefit of combining LSPM with KV-cache-side management techniques such as StreamingLLM [20] or H2O [21], as discussed in Section 6. Finally, we plan to extend the dynamic compression-ratio controller described in Section 3.4 beyond linear interpolation to a learned policy trained on observed load patterns, and to evaluate its stability and responsiveness under realistic, bursty production traffic rather than the synthetic load profiles used in initial systems testing.

9. Conclusion

Arabic RAG systems deployed on memory-efficient serving engines such as vLLM face a KV-cache pressure problem rooted in tokenization: Arabic subword fragmentation inflates the token count of retrieved

context relative to semantically equivalent English content, which we confirm directly across two independent tokenizers at an aggregate ratio of 1.69x–1.76x on a hand-verified parallel corpus. We introduced LSPM, a lightweight, sentence-level, cross-encoder-driven pruning middleware that sits between retrieval and generation in an Arabic RAG pipeline and can either apply a fixed compression ratio or adapt dynamically to vLLM's live KV-cache occupancy. In a real, live-endpoint preliminary pilot, 50% context compression preserved downstream answer fidelity by a substantial margin across three complementary automatic metrics (ROUGE-L 0.928, BERTScore-F1 0.971, BLEU 81.9). We have been explicit throughout that the systems-level throughput and KV-cache-savings claim central to this work's motivation remains to be measured on GPU-scale infrastructure, and we have released the complete implementation, evaluation harness, and benchmarking protocol needed to close that gap, together with a live demonstration interface, in the interest of full reproducibility.

Declarations

Data availability. The 30-pair Arabic-English tokenization corpus, the 8-question/10-document fidelity pilot dataset, all raw experimental outputs (tokenization CSVs, raw/pruned answer pairs, fidelity metric scores), and the complete source code (middleware, evaluation scripts, benchmarking harness, and demonstration interface) are publicly available in the project's GitHub repository at the URL provided in the code and data availability statement accompanying this submission.

Ethics declaration. This study did not involve human participants, personal data, or clinical data. The parallel-corpus and pilot-corpus text was authored by the researcher for evaluation purposes and describes only publicly available, non-sensitive institutional information. No ethics committee approval was required.

Author contributions (CRediT). Akram Taha: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

Conflict of interest. The author declares no conflict of interest.

Funding. This research received no external funding. Inference costs for the preliminary experiments were incurred against a free-tier hosted API allowance.

AI usage disclosure. Large language model assistance (Anthropic Claude) was used during this project for software implementation (the LSPM middleware, evaluation scripts, and demonstration interface), for literature search assistance in identifying candidate related work subsequently verified by the author against primary sources, and for drafting assistance on this manuscript under the author's direction and review. All experimental results reported in Section 5 were produced by executing the released code against live infrastructure as described in Section 4; no experimental results were generated or estimated by the language model without corresponding code execution. The author takes full responsibility for the accuracy of all claims, citations, and reported results in this paper.

References

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2020.

[2] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," in Proc. 29th ACM Symp. Operating Systems Principles (SOSP), 2023, doi: 10.1145/3600006.3613165.

[3] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2023.

[4] Y. Li, B. Dong, C. Lin, and F. Guerin, "Compressing Context to Enhance Inference Efficiency of Large Language Models," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2023.

[5] F. Xu, W. Shi, and E. Choi, "RECOMP: Improving Retrieval-Augmented LMs with Compression and Selective Augmentation," in Proc. Int. Conf. Learning Representations (ICLR), 2024.

[6] O. Khattab and M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," in Proc. 43rd Int. ACM SIGIR Conf. Research and Development in Information Retrieval, 2020.

[7] R. Nogueira and K. Cho, "Passage Re-ranking with BERT," arXiv preprint arXiv:1901.04085, 2019.

[8] BAAI FlagEmbedding Team, "BGE-M3: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings," BAAI Technical Report, 2024. [Online]. Available: https://huggingface.co/BAAI/bge-reranker-v2-m3

[9] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks," in Proc. Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing (EMNLP-IJCNLP), 2019.

[10] V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2020.

[11] W. Antoun, F. Baly, and H. Hajj, "AraBERT: Transformer-based Model for Arabic Language Understanding," in Proc. 4th Workshop on Open-Source Arabic Corpora and Processing Tools (OSACT), 2020.

[12] H. Mozannar, K. El Hajal, E. Maamary, and H. Hajj, "Neural Arabic Question Answering," in Proc. 4th Arabic Natural Language Processing Workshop (WANLP), 2019.

[13] N. Sengupta et al., "Jais and Jais-chat: Arabic-Centric Foundation and Instruction-Tuned Open Generative Large Language Models," arXiv preprint arXiv:2308.16149, 2023.

[14] C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," in Text Summarization Branches Out: Proc. ACL Workshop, 2004, pp. 74–81.

[15] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A Method for Automatic Evaluation of Machine Translation," in Proc. 40th Annual Meeting of the Association for Computational Linguistics (ACL), 2002, pp. 311–318.

[16] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating Text Generation with BERT," in Proc. Int. Conf. Learning Representations (ICLR), 2020.

[17] Qwen Team, "Qwen2.5 Technical Report," arXiv preprint arXiv:2412.15115, 2024.

[18] AI@Meta, "The Llama 3 Herd of Models," arXiv preprint arXiv:2407.21783, 2024.

[19] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2022.

[20] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient Streaming Language Models with Attention Sinks," in Proc. Int. Conf. Learning Representations (ICLR), 2024.

[21] Z. Zhang et al., "H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2023.

[22] Y. Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv preprint arXiv:2312.10997, 2023.

[23] A. Vaswani et al., "Attention Is All You Need," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2017.

[24] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," IEEE Transactions on Big Data, vol. 7, no. 3, pp. 535–547, 2021.

[25] R. Sennrich, B. Haddow, and A. Birch, "Neural Machine Translation of Rare Words with Subword Units," in Proc. 54th Annual Meeting of the Association for Computational Linguistics (ACL), 2016, pp. 1715–1725.

[26] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A Distributed Serving System for Transformer-Based Generative Models," in Proc. 16th USENIX Symp. Operating Systems Design and Implementation (OSDI), 2022.

[27] T. B. Brown et al., "Language Models are Few-Shot Learners," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2020.

[28] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," in Proc. Int. Conf. Learning Representations (ICLR), 2024.

[29] S. Es, J. James, L. Espinosa Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," in Proc. 18th Conf. European Chapter of the Association for Computational Linguistics: System Demonstrations (EACL), 2024.

[30] N. F. Liu et al., "Lost in the Middle: How Language Models Use Long Contexts," Transactions of the Association for Computational Linguistics, vol. 12, pp. 157–173, 2024.

[31] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, "Multilingual E5 Text Embeddings: A Technical Report," arXiv preprint arXiv:2402.05672, 2024.

[32] X. Zhang et al., "MIRACL: A Multilingual Retrieval Dataset Covering 18 Diverse Languages," Transactions of the Association for Computational Linguistics, vol. 11, pp. 1114–1131, 2023.

[33] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," in Proc. Int. Conf. Learning Representations (ICLR), 2023.

[34] O. Obeid et al., "CAMeL Tools: An Open Source Python Toolkit for Arabic Natural Language Processing," in Proc. 12th Language Resources and Evaluation Conf. (LREC), 2020, pp. 7022–7032.

[35] W. Antoun, F. Baly, and H. Hajj, "AraGPT2: Pre-Trained Transformer for Arabic Language Generation," in Proc. 6th Arabic Natural Language Processing Workshop (WANLP), 2021.

[36] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: Massive Text Embedding Benchmark," in Proc. 17th Conf. European Chapter of the Association for Computational Linguistics (EACL), 2023.

[37] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009.

[38] J. Lin et al., "AWQ: Activation-Aware Weight Quantization for LLM Compression and Acceleration," in Proc. Machine Learning and Systems (MLSys), 2024.

[39] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," in Proc. 40th Int. Conf. Machine Learning (ICML), 2023.

[40] A. Huang et al., "ACVA: Arabic Cultural and Value Alignment Benchmark," Hugging Face Dataset, 2024. [Online]. Available: https://huggingface.co/datasets/FreedomIntelligence/ACVA-Arabic-Cultural-Value-Alignment

[41] F. Koto et al., "ArabicMMLU: Assessing Massive Multitask Language Understanding in Arabic," 2024.